1 Random Sized Recursion

Problem 1.1. Consider the following algorithm,

(i) Determine the **best case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(ii) Determine the **worst case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(iii) Determine the expected case running time of the algorithm. **Fully justify your solution.**

Assume that Random(n) takes constant time and returns a number between 1 and n uniformly randomly.

```
function Func(A[],n)
    if n ≤ 30 then
        return 1
    end if
    x = 0
    for i = 1 to n do
        for j = 1 to n/3 do
            x = x + A[i] − A[j]
        end for
    end for
    k = Random(n − 37)
    x = x + Func(A[], k)
    return x
end function
```

cn^2

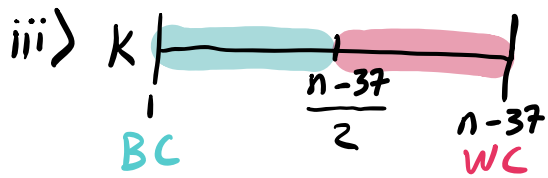
i) BC occurs when $k=1$ first time.

$B(n) = cn^2 + B(1) \Rightarrow B(n) \in \Theta(n^2)$

ii) WC occurs when $k = n-37$ always.

$W(n) = cn^2 + W(n-37) \Rightarrow W(n) \in \Theta(n^3)$

iii)



$$ET(n) = E\left(T(n) \mid k \le \tfrac{n-37}{2}\right) Pr\left(k \le \tfrac{n-37}{2}\right) + E\left(T(n) \mid k > \tfrac{n-37}{2}\right) Pr\left(k > \tfrac{n-37}{2}\right)$$

$$ET(n) \le \left[cn^2 + ET\left(\tfrac{n-37}{2}\right)\right]\left(\tfrac{1}{2}\right) + \left[cn^2 + ET(n-37)\right]\left(\tfrac{1}{2}\right)$$

$$ET(n) \le cn^2 + \tfrac{1}{2} ET\left(\tfrac{n-37}{2}\right) + \tfrac{1}{2} ET(n-37)$$

$$ET(n) \le cn^2 + \tfrac{1}{2} ET\left(\tfrac{n}{2}\right) + \tfrac{1}{2} ET(n) \qquad \text{let } \alpha = 2c$$

$$\tfrac{1}{2} ET(n) \le cn^2 + \tfrac{1}{2} ET(n/2)$$

$$ET(n) \le 2cn^2 + ET(n/2) = \alpha n^2 + ET(n/2)$$

$$ET(n) \le \sum_{i=0}^{h} \alpha (n/2^i)^2 \le \alpha n^2 \sum_{i=0}^{\infty} (1/4)^i = \alpha n^2 \left(\frac{1}{1-1/4}\right) = \frac{4}{3}\alpha n^2.$$

Thus, $ET(n) \in O(n^2)$.

TREE!

αn^2

$\alpha (n/2)^2$

$\alpha (n/4)^2$

$\vdots$

$ET(30)$

h

$\frac{n}{2^h} = 30$

$h = \log_2(n/30)$

The BC is a LB, so $ET(n) \in \Omega(n^2)$.

THEREFORE: $ET(n) \in \Theta(n^2)$.

Problem 1.2. Consider the following algorithm,

(i) Determine the **best case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(ii) Determine the **worst case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(iii) Determine the expected case running time of the algorithm. **Fully justify your solution.**

Assume that $\texttt{Random}(n)$ takes constant time and returns a number between 1 and n uniformly randomly. Assume that the more uneven the split between the recursive calls, the longer the running time of the function.

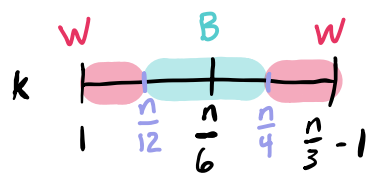
```
function Func(A[],n)
    if n ≤ 22 then
        return 1
    end if
    x = 0
    for i = 1 to n/2 do
        for j = 1 to n/4 do
            x = x + A[i] − A[j]
        end for
    end for
    k = Random(n/3 − 1)
    x = x + Func(A[], 3k)
    x = x + Func(A[], n − 3k)
    return x
end function
```

i) BC occurs when $k = \frac{n}{6}$ ALWAYS.

$B(n) = cn^2 + 2B(n/2) \Rightarrow B(n) \in \Theta(n^2)$.

ii) WC occurs when $k = 1$ OR $\frac{n}{3} - 1$ ALWAYS.

$W(n) = cn^2 + W(n-3) + W(3) \Rightarrow W(n) \in \Theta(n^3)$.



$$ET(n) \le E\left(T(n) \mid k \le \tfrac{n}{12}\right) Pr\left(k \le \tfrac{n}{12}\right) + E\left(T(n) \mid \tfrac{n}{12} < k \le \tfrac{n}{4}\right) Pr\left(\tfrac{n}{12} < k \le \tfrac{n}{4}\right) + E\left(T(n) \mid k > \tfrac{n}{4}\right) Pr\left(k > \tfrac{n}{4}\right)$$

UB:

$$ET(n) \le \frac{1}{4}\left[cn^2 + ET(n-3) + ET(3)\right] + \frac{1}{2}\left[cn^2 + ET\left(n - 3(n/12)\right) + ET\left(3(n/12)\right)\right] + \frac{1}{4}\left[cn^2 + ET\left(n - 3\left(\tfrac{n}{3} - 1\right)\right) + ET\left(3\left(\tfrac{n}{3} - 1\right)\right)\right]$$

$$ET(n) \le cn^2 + \frac{1}{4}ET(n-3) + \frac{1}{4}ET(n-3) + \frac{1}{2}ET(3n/4) + \frac{1}{2}ET\left(\frac{n}{4}\right)$$

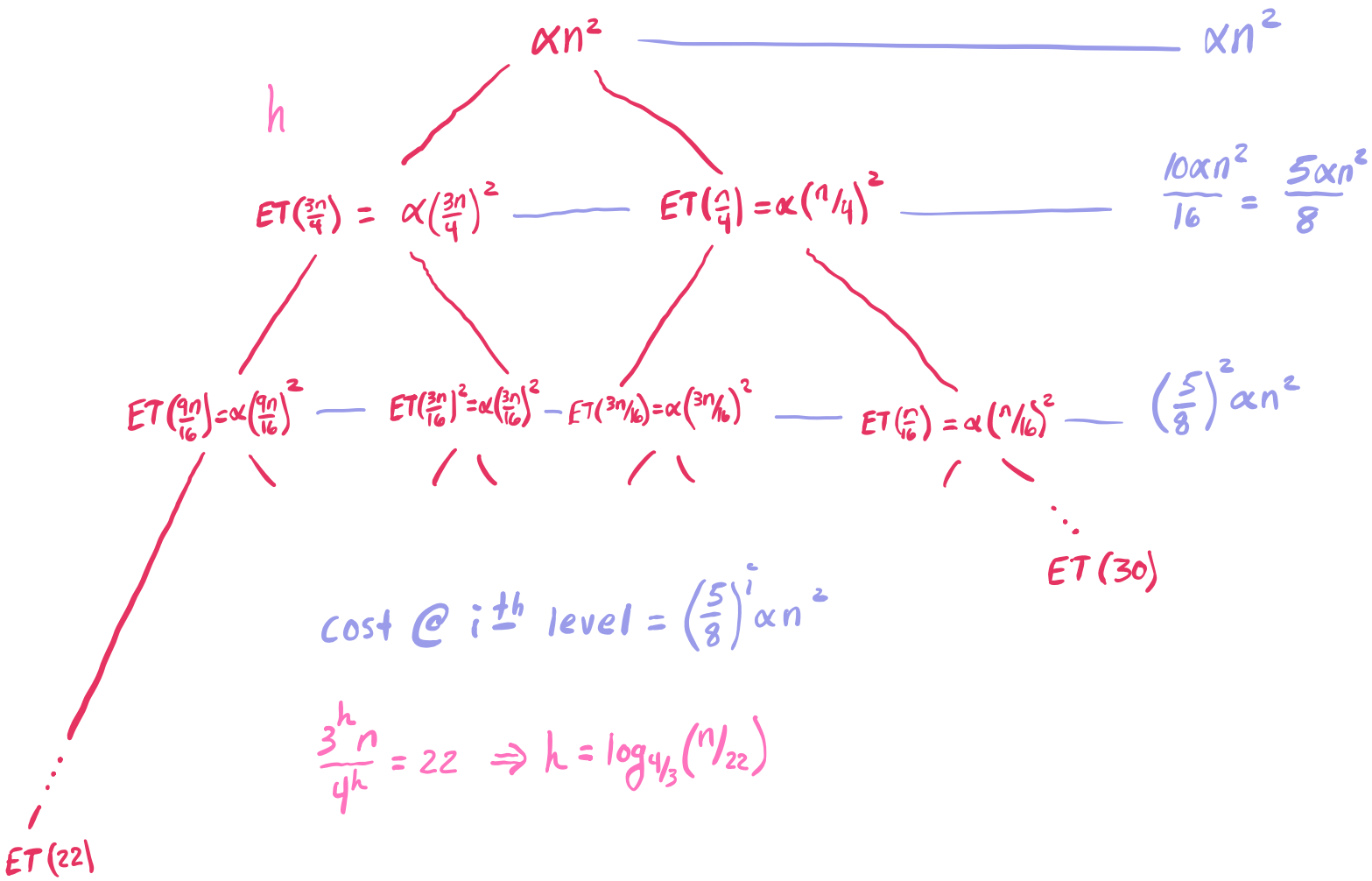
$$ET(n) \le cn^2 + \frac{1}{4}ET(n) + \frac{1}{4}ET(n) + \frac{1}{2}ET(3n/4) + \frac{1}{2}ET(n/4)$$

$$\frac{1}{2}ET(n) \le cn^2 + \frac{1}{2}ET(3n/4) + \frac{1}{2}ET(n/4)$$

let $\alpha = 2c$

$$ET(n) \le 2cn^2 + ET(3n/4) + ET(n/4)$$

$$ET(n) \le \alpha n^2 + ET(3n/4) + ET(n/4)$$



cost @ i^{th} level $= \left(\frac{5}{8}\right)^i \alpha n^2$

$$\frac{3^h n}{4^h} = 22 \Rightarrow h = \log_{4/3}(n/22)$$

$$\left.\begin{aligned}
ET(n) &\le \sum_{i=0}^{h} \alpha n^2 (5/8)^i \\
ET(n) &\le \alpha n^2 \sum_{i=0}^{\infty} (5/8)^i \\
ET(n) &\le \alpha n^2 \left[\frac{1}{1 - 5/8}\right] \\
ET(n) &\le \alpha n^2 [8/3]
\end{aligned}\right\} O(n^2)$$

$ET(n) \ge \alpha n^2$

Since the loop on lines 6-10 takes n^2 time.

Thus, $ET(n) \in \Theta(n^2)$.

2 Casino Problems

Problem 2.1. Consider the following algorithm,

(i) Determine the **best case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(ii) Determine the **worst case running time** of the algorithm and **state what condition must be true** for it to occur. No justification is needed!

(iii) Determine the expected case running time of the algorithm. **Fully justify your solution.**

Assume that $\texttt{Random}(n)$ takes constant time and returns a number between 1 and n uniformly randomly.

(a)

```
function Func(A[],n)
    x = 0
    for i = 1 to n^2 do
        k = Random(n)
        if k ≤ n/2 then
            return x
        end if
        for i = 1 to n do
            x = x + A[i]
        end for
    end for
    return x
end function
```

i) BC occurs when $k \le n/2$ FIRST TIME.
$B(n) \in \Theta(1)$.

ii) WC occurs when $k > n/2$ ALWAYS.
$W(n) \in \Theta(n^3)$.

iii) Let X = # of times $k > n/2$

$T(n) = cn \cdot X + c$

$ET(n) = c + cn\,E(X)$

$ET(n) = c + cn \sum_{i=1}^{n^2} Pr(X \ge i)$

$ET(n) = c + cn \sum_{i=1}^{n^2} (1/2)^i$

$Pr(k > n/2) = 1/2$

$Pr(X \ge 1) = 1/2$

$Pr(X \ge 2) = \left(\frac{1}{2}\right)^2$

$Pr(X \ge i) = \left(\frac{1}{2}\right)^i$

UB:

$ET(n) \le c + cn \sum_{i=0}^{\infty} (1/2)^i = c + cn\left(\frac{1}{1-1/2}\right) = c + cn$

LB:

$ET(n) \ge \frac{cn}{2}$

Thus, $ET(n) \in \Theta(n)$.

(b)

```
function Func(A[],n)
    x = 0
    k = Random(n)
    repeat
        k = Random(n)
        for i = 1 to ⌊log_2(n)⌋ do
            x = x + A[i]
        end for
    until k = 42
    return x
end function
```

i) BC occurs when $k = 42$ FIRST TIME.

$B(n) \in \Theta(\log(n))$

ii) WC occurs when k NEVER $= 42$

$W(n) \in O(\infty)$

iii) Let $X = \#$ of times $k \neq 42$

$$T(n) = c\log(n) \cdot X + c\log(n)$$

$$ET(n) = c\log(n) + c\log(n)\,E(X)$$

$$ET(n) = c\log(n) + c\log(n)\sum_{i=1}^{\infty} Pr(X \geq i)$$

$$ET(n) = c\log(n) + c\log(n)\sum_{i=1}^{\infty} \left(\frac{n-1}{n}\right)^i$$

$$ET(n) = c\log(n) + c\log(n)\left[\sum_{i=0}^{\infty}\left(\frac{n-1}{n}\right)^i - 1\right]$$

$$ET(n) = c\log(n) + c\log(n)\left[\frac{1}{1-\frac{n-1}{n}} - 1\right]$$

$$ET(n) = c\log(n) + c\log(n)\left[\frac{n}{n-(n-1)} - 1\right]$$

$$ET(n) = c\log(n) + c\log(n)\left[n - 1\right]$$

$Pr(k \neq 42) = \frac{n-1}{n}$

$Pr(X \geq 1) = \frac{n-1}{n}$

$Pr(X \geq 2) = \left(\frac{n-1}{n}\right)^2$

$Pr(X \geq i) = \left(\frac{n-1}{n}\right)^i$

Thus, $ET(n) \in \Theta(n\log(n))$

(c)

```
function Func(n)
    B is a bin of n orange balls and 3n yellow balls.
    repeat
        β = select and remove a random ball from B
        if β is orange then
            Print "Success!"
        else
            for i = 1 to n do
                Print "Failure."
            end for
        end if
    until β is orange
end function
```

cn

i) BC occurs when first ball is ORANGE

$B(n) \in \Theta(1)$

ii) WC occurs when all $3n$ yellow balls are removed before any orange. $W(n) \in \Theta(n^2)$.

iii) Let X = # of yellow balls removed.

$T(n) = c\,n \cdot X + c$

$ET(n) = c + cn\,E(X)$

$ET(n) = c + cn \sum_{i=1}^{3n} Pr(X \geq i)$

$Pr(X \geq 1) = Pr(\text{first ball yellow}) = \frac{3n}{4n} = \frac{3}{4}$

$Pr(X \geq 2) = Pr(\text{first 2 yellow}) = \frac{3n}{4n} \cdot \frac{3n-1}{4n-1} \leq \left(\frac{3}{4}\right)^2$

$Pr(X \geq 3) = Pr(\text{first 3 yellow}) = \frac{3n}{4n} \cdot \frac{3n-1}{4n-1} \cdot \frac{3n-2}{4n-2} \leq \left(3/4\right)^3$

$Pr(X \geq i) \leq \left(3/4\right)^i$

$= \prod_{K=0}^{i-1} \frac{3n-K}{4n-K}$

UB:

$ET(n) \leq c + cn \sum_{i=0}^{\infty} \left(3/4\right)^i$

$ET(n) \leq c + cn\left[\frac{1}{1-3/4}\right] = c + 4cn$

LB:

$ET(n) = c + cn\left(\frac{3}{4} + \dots\right)$

$ET(n) \geq cn \cdot \frac{3}{4}$

Thus, $ET(n) \in \Theta(n)$.

3 Quick Sort and Selection

Problem 3.1. Determine the **expected running time** of the algorithm given below. Assume that,

- The best case running time of this algorithm is $\Theta(n\log(n))$.
- The worst case running time is $\Theta(n^2)$.

where $n = j - i + 1$.

Assume that $\texttt{Random}(n)$ uniformly randomly returns integer values between 1 and n.

```
function Func(A[],i,j)
    n = j - i + 1
    if n ≤ 1 then
        return 1
    end if
    x = 0
    for m = i to j do
        x = x + m
    end for
    k = Random(n - 1)
    x = x + Func(A, i, i + k - 1)
    x = x + Func(A, i + k, j)
    return x
end function
```

cn

size $= i + k - 1 - i + 1 = k$

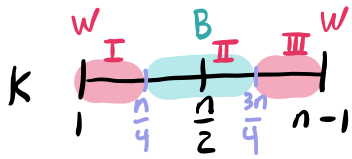
size $= j - (i+k) + 1 = j - i + 1 - k = n - k$

$$T(n) = cn + T(k) + T(n-k).$$

The best case occurs when the calls are as evenly split as possible. That is,

$k = n - k$

$k = n/2$



$$ET(n) = Pr(\mathrm{I})\,E(T(n) \mid \mathrm{I}) + Pr(\mathrm{II})\,E(T(n) \mid \mathrm{II}) + Pr(\mathrm{III})\,E(T(n) \mid \mathrm{III})$$

$$ET(n) \le \frac{1}{4}\left(cn + \cancel{ET(1)}^{\,c} + \underbrace{ET(n-1)}_{\le ET(n)}\right) + \frac{1}{2}\left(cn + ET\left(\tfrac{n}{4}\right) + ET\left(\tfrac{3n}{4}\right)\right) + \frac{1}{4}\left(cn + \underbrace{ET(n-1)}_{\le ET(n)} + \cancel{ET(1)}^{\,c}\right)$$

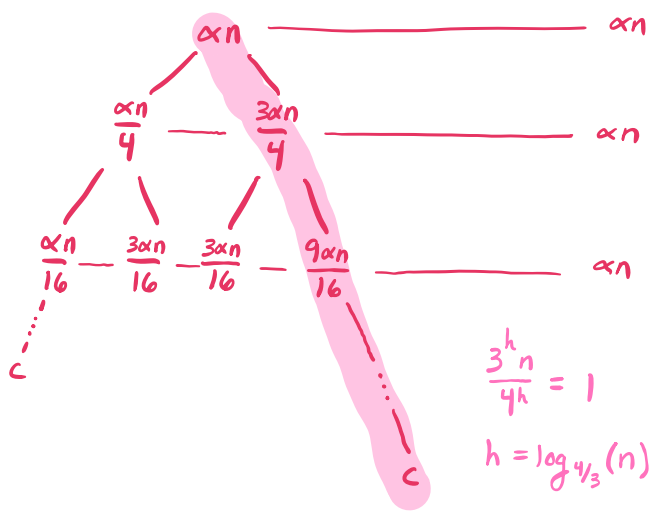
$$ET(n) \le cn + \frac{1}{2}ET(n) + \frac{1}{2}ET(n/4) + \frac{1}{2}ET(3n/4)$$

$$\frac{1}{2}ET(n) \le cn + \frac{1}{2}ET(n/4) + \frac{1}{2}ET(3n/4)$$

$$ET(n) \le 2cn + ET(n/4) + ET(3n/4)$$

$$ET(n) \le \alpha n + ET(n/4) + ET(3n/4)$$

let $\alpha = 2c$



$$\frac{3^h n}{4^h} = 1$$

$$h = \log_{4/3}(n)$$

$$ET(n) \le \alpha n \cdot \log_{\frac{4}{3}}(n)$$

Thus, $ET(n) \in O(n\log(n))$

The best case is always a lower bound, so $ET(n) \in \Omega(n\log(n))$.

Therefore, $ET(n) \in \Theta(n\log(n))$.

Problem 3.2. We have discussed four sorting algorithms at this point in the class, `SelectionSort`, `MergeSort`, `InsertionSort`, and `QuickSort`. For each of these algorithms,

(a) Explain what the algorithm does/how the algorithm works.

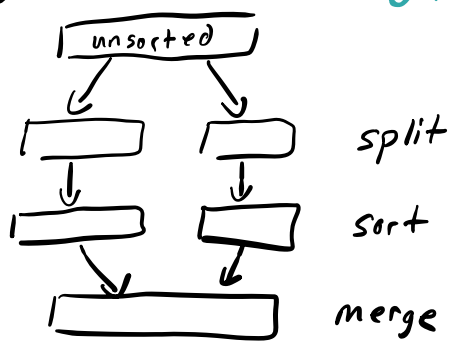
(b) What is the running time? (DO NOT DERIVE THEM, JUST WRITE THEM DOWN).

(c) Does the algorithm have any benefits (when compared to the others)?

(d) Does the algorithm have any drawbacks (when compared to the others)?

Merge Sort

$\Theta(n\log(n))$



Useful because it is consistently fast.

GREAT on Linked Lists.

Selection Sort

Find the max and place it at the start, repeat recursively.

$\Theta(n^2)$. SLOW!

Only "redeeming" quality is that it is exceptionally easy to code.

Insertion Sort

Iteratively take one element from the unsorted part and correctly insert it into the sorted part.

sorted	unsort

WC: $\Theta(n^2)$

BC: $\Theta(n)$

EC: $\Theta(n^2)$

FAST best case.

SLOW worst case.

great when: n small

A nearly sorted

Quick Sort

AMAZING

BAD worst case.

Partition around a pivot and sort recursively.

$\leq p$	p	$> p$

BC / EC: $\Theta(n\log(n))$

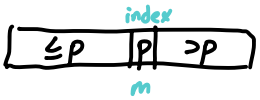
WC: $\Theta(n^2)$

How to pick a pivot

1) Random
2) First element / arbitrary.
3) Median
4) Average
5) Average of 3
6) Median of 3

Problem 3.3. In the notes, we discussed an algorithm called `RandomizedSelect`. Explain what this algorithm does and why it might be used.

Randomized selection is an algorithm for finding the kth order statistic. It works by partitioning an array around a pivot.



After this partition, p is in it's correct index. If $k < m$, search left
If $k > m$, search right
If $k = m$, Yay! Done!